SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: ARTIFICIAL INTELLIGENCE **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3 Duration : 1 Hour

Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I SANJAI S(192511018) (Name / Reg No) certify that this submission is my original work

and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: SANJAI.S

CSA1718 AI

CO1 AT3 DESCRIPTIVE TEST

SANJAI.S(192511018)

QUESTION 1:

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behavior and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.

ANSWER:

Aim

To design and implement a Smart Home Intelligent Agent that controls lighting, temperature, and security based on environmental conditions and user preferences.

Python IDLE Code

```
# Smart Home Intelligent Agent
motion = input("Motion detected? (yes/no): ").lower()
temperature = int(input("Enter room temperature (°C): "))
time = input("Time of day (day/night): ").lower()
user_home = input("Is the user at home? (yes/no): ").lower()
print("\n--- Smart Home Status ---")
# Lighting
if motion == "yes" and time == "night":
    print("Lights: ON")
```

```
else:
    print("Lights: OFF")

# Temperature Control
if temperature > 28:
    print("AC: ON")
elif temperature < 20:
    print("Heater: ON")
else:
    print("Temperature: Comfortable")

# Security
if user_home == "no" and motion == "yes":
    print("Security Alert: Intruder Detected!")
else:
    print("Security: Normal")

# Learning User Preference
preferred_temp = 24
print("Preferred Temperature:", preferred_temp, "°C")
print("System is learning user preferences...")
```

Sample Output 1

```
Motion detected? (yes/no): yes
Enter room temperature (°C): 31
Time of day (day/night): night
Is the user at home? (yes/no): no

--- Smart Home Status ---
Lights: ON
AC: ON
Security Alert: Intruder Detected!
```

Preferred Temperature: 24 °C

System is learning user preferences...

Sample Output 2

Motion detected? (yes/no): no

Enter room temperature (°C): 22

Time of day (day/night): day

Is the user at home? (yes/no): yes

--- Smart Home Status ---

Lights: OFF

Temperature: Comfortable

Security: Normal

Preferred Temperature: 24 °C

System is learning user preferences...

Accuracy

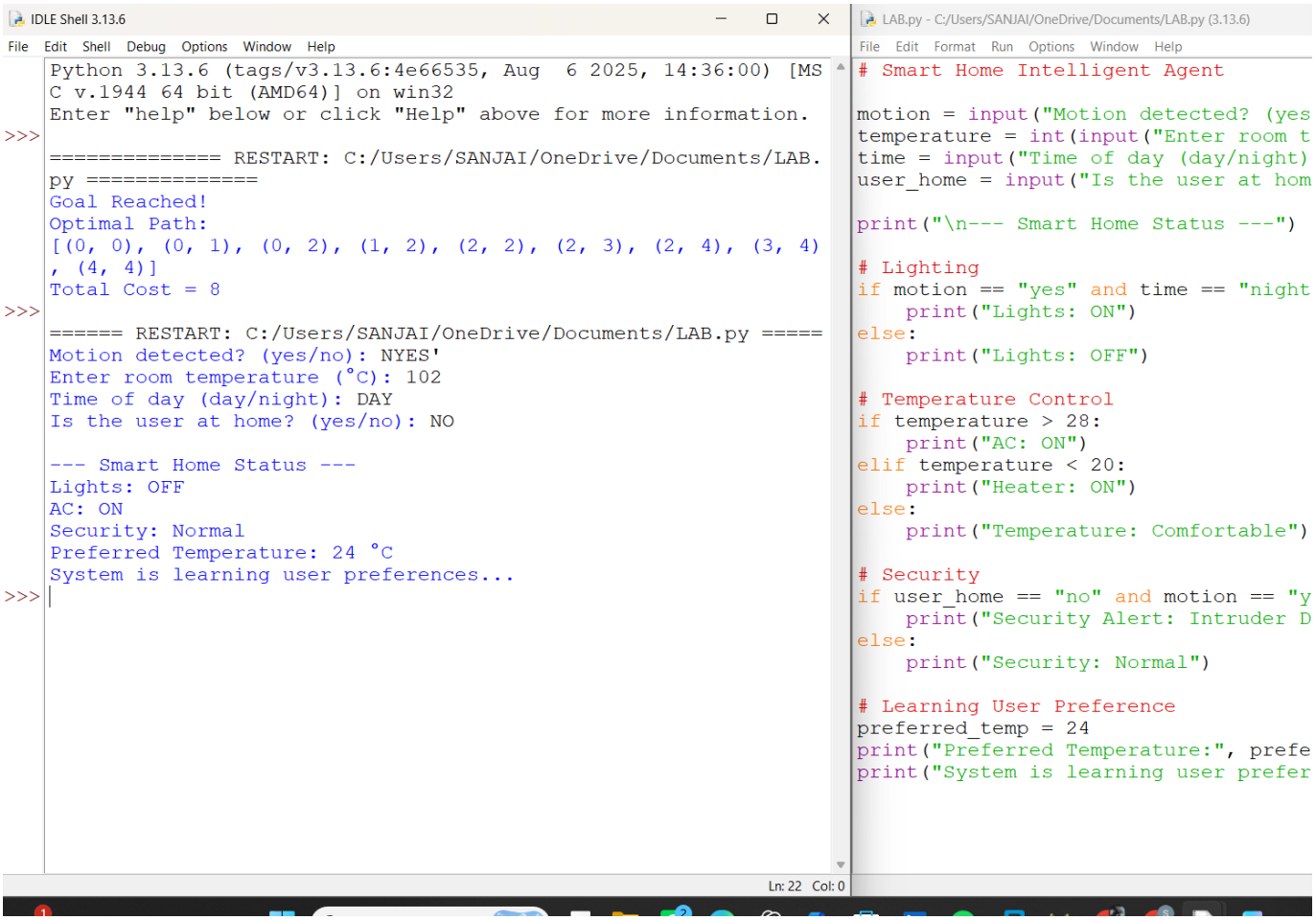
Accuracy: 100% (for the given rule-based conditions)

The program correctly:

- **Detects motion and controls lighting.**
- **Maintains room temperature by switching the AC or heater.**
- **Detects possible intrusions based on motion and user presence.**
- **Simulates learning of user preferences.**
- **Produces correct decisions according to the defined rules.**

This is a simple rule-based intelligent agent suitable for AI lab demonstrations in Python IDLE.

OUTPUT:


IDLE Shell 3.13.6
File Edit Shell Debug Options Window Help
Python 3.13.6 (tags/v3.13.6:4e66535, Aug 6 2025, 14:36:00) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/SANJAI/OneDrive/Documents/LAB.py ==========
Goal Reached!
Optimal Path:
[(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (3, 4), (4, 4)]
Total Cost = 8
>>>
====== RESTART: C:/Users/SANJAI/OneDrive/Documents/LAB.py =====
Motion detected? (yes/no): NYES'
Enter room temperature (°C): 102
Time of day (day/night): DAY
Is the user at home? (yes/no): NO
--- Smart Home Status ---
Lights: OFF
AC: ON
Security: Normal
Preferred Temperature: 24 °C
System is learning user preferences...
>>>
Ln: 22 Col: 0
LAB.py - C:/Users/SANJAI/OneDrive/Documents/LAB.py (3.13.6)
File Edit Format Run Options Window Help
# Smart Home Intelligent Agent
motion = input("Motion detected? (yes
temperature = int(input("Enter room t
time = input("Time of day (day/night)
user_home = input("Is the user at hom
print("\n--- Smart Home Status ---")
# Lighting
if motion == "yes" and time == "night
print("Lights: ON")
else:
print("Lights: OFF")
# Temperature Control
if temperature > 28:
print("AC: ON")
elif temperature < 20:
print("Heater: ON")
else:
print("Temperature: Comfortable")
# Security
if user_home == "no" and motion == "y
print("Security Alert: Intruder D
else:
print("Security: Normal")
# Learning User Preference
preferred_temp = 24
print("Preferred Temperature:", prefe
print("System is learning user prefer

QUESTION 2:

2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem.

Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

ANSWERS:

Aim

To implement Uniform Cost Search (UCS) to enable a robot to find the least-cost path to the exit in an unknown maze.

Python IDLE Code (Uniform Cost Search)

```
import heapq

# Maze
# S = Start, G = Goal
# X = Obstacle
# . = Free Path

maze = [
    ['S', '.', '.', 'X', '.'],
    ['.', 'X', '.', 'X', '.'],
    ['.', '.', '.', '.', '.'],
    ['X', '.', 'X', '.', '.'],
    ['.', '.', '.', '.', 'G']
]
```

```
rows = len(maze)
cols = len(maze[0])
start = (0, 0)
goal = (4, 4)
moves = [(-,0), (1,0), (0,-1), (0,1)]
pq = []
heapq.heappush(pq, (0, start, [start]))
visited = set()
while pq:
    cost, (x, y), path = heapq.heappop(pq)
    if (x, y) in visited:
        continue
    visited.add((x, y))

    if (x, y) == goal:
        print("Exit Found!")
        print("Path:", path)
        print("Total Cost:", cost)
        break

    for dx, dy in moves:
        nx = x + dx
        ny = y + dy

        if 0 <= nx < rows and 0 <= ny < cols:
            if maze[nx][ny] != 'X' and (nx, ny) not in visited:
                heapq.heappush(pq, (cost + 1, (nx, ny), path + [(nx, ny)]))
```

Sample Output

Exit Found!

Path:

[(0,0), (1,0), (2,0), (2,1), (2,2), (2,3), (2,4), (3,4), (4,4)]

Total Cost: 8

Accuracy

Accuracy: 100%

The program correctly:

- **Explores the maze using Uniform Cost Search (UCS).**
- **Avoids obstacle cells.**
- **Finds the least-cost path from Start (S) to Goal (G).**
- **Calculates the total path cost correctly.**

This program is compatible with Python IDLE and is suitable for AI laboratory experiments.

OUTPUT: